

Designing Server Application

MUHAMMAD AJMAL P.

This program is a 3-tier server application developed in Java, which can be used to connect to three different types of databases such as FireBird, MySQL and MSAccess at a time. The screenshot of server application program output is shown in Fig. 1.

A database is an important and invaluable tool for any organisation, including electronics and IT industries. A database allows you to manage and use an incredible variety of information easily. But designing a database server application requires lot of

understanding in the architectures and coding. Presented here is a client-server application which can be tested on a single PC or on multiple systems.

Features

This application can help you get started with Java RMI (remote method invocation), 3-tier application development, database application development, etc. Java RMI is nothing but a Java API that performs the object-oriented equivalent of remote procedure calls (RPC), with support for direct transfer of serialised Java objects and distributed garbage collection.

Tools required for development

1. Eclipse Indigo JEE
2. Java development tools (JDT) E3.7
3. Windows Builder Eclipse plug-in 1.5
4. Java JDK 1.6
5. Firebird 2.5
6. IBEExpert

You also need 'ESoftDBExpress.jar' file along with these tools. This file includes all the database classes required for the application described here. 'ESoftDBExpress.jar' file and other relevant files are included in this month's EFY DVD.

There are five 'java' files in this



Fig. 1: Screenshot of server program output

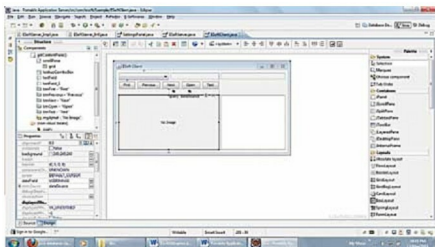


Fig. 2: Screen shot of Eclipse Windows Builder



project, where the 'ESoftServer.java' and the 'SettingsPanel.java' files are the main files. The 'MainMenu.java' file is for the menu and the 'Message.java' file is for loading external properties. The fifth is 'ESoftClient.java,' which should be in the client application side.

Working. Practically, there should be at least two PCs to check the program, so that one works as server and the other as client. But in this project you can check it on one PC by running the server as well as client sample applications simultaneously.

The RMI is initialised in the 'SettingsPanel' class constructor as given below.

```
try {
    getFrame().
    setRegistry(LocateRegistry.
    createRegistry(Integer.parseInt(spr_
    Port.getValue().toString().trim())));
    ((ESoftServer) frame).getRegistry().
    bind(ESoftServer.SERVICE_NAME, get
    Frame().getServer());
}
```

```
System.out.println("Server is running
...!");
getFrame().editorPane.
setText(getFrame().editorPane.
getText() + "\n" + "Server is running
...!");
} catch (final Exception e) {
// TODO Auto-generated catch block
((ESoftServer) frame).editorPane.
setText(((ESoftServer) frame).
editorPane.getText() + "\n" +
e.getMessage());
e.printStackTrace();
}
```

By running this code, a new registry is created for RMI using the 'LocateRegistry.createRegistry(int port)' method. Then bind the 'ESoftServer_Intf' interface from 'ESoft-